第八章 持久性和数据库设计

2018年11月20日 8:28

1. 业务对象与数据库设计
   - 实体类表示应用程序中的持久数据库对象，但实体类不是数据库中的持久类。称它们持久是因为在应用程序终止之前，实体对象的最新映射将会持久保存在数据库中。

   a. 数据库管理系统
      - 数据库可以是关系型、基于关系演算的数据库模型
      - 对象-关系型
      - 面向对象型

   b. 数据模型的层次
      - 过去数据库主要把精力放在**数据模型**（UML称为静态模型）上。当今数据库存储和执行程序的功能已经扩展到**行为模型**（以触发器和存储过程为中心）。
      - **数据模型**（也称为数据库模式）

      i. 外部（概念）数据模型
         1) 实体关系（ER）图——最流行的概念数据建模技术

      ii. 逻辑数据模型
         1) 逻辑模式，也称为全局概念模式
         2) 能够反映系统实现时所采用的数据库模型的逻辑存储结构（表等）的模型

      iii. 物理数据模型
         1) 专门针对特定的DBMS
         2) 定义数据如何真正存储在持久存储设备上，通常指磁盘

   c. 集成应用系统与数据库建模

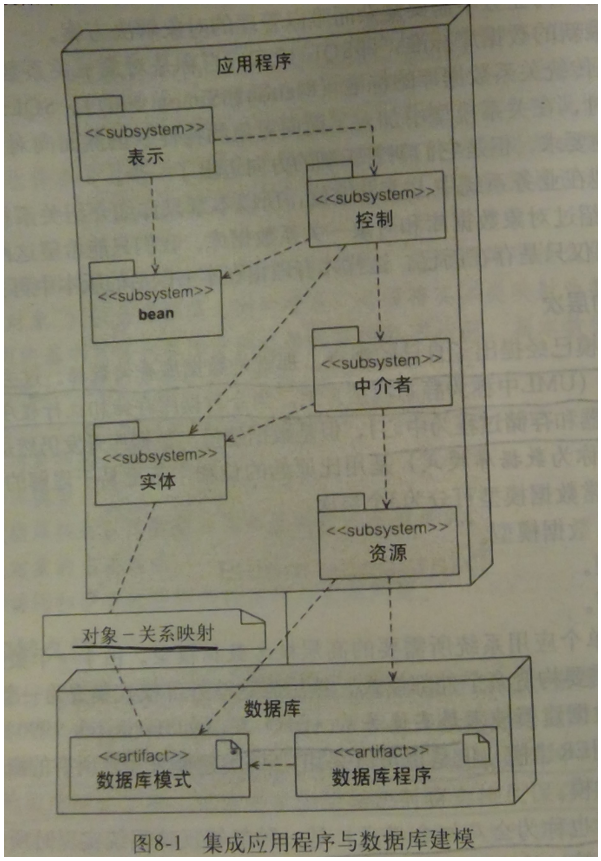
i. 

图8-1 集成应用程序与数据库建模

ii. PCBMER中的向下依赖原则DDP扩展了应用程序与数据库间的通信

iii. 面向对象数据库模型可以用作客户端应用程序和服务器端数据源之间协同工作的对象存储API

d. 对象-数据库映射基础

i. 映射困难的原因：

1) 数据库的存储结构处理不了面向对象泛型

2) 数据库几乎都不是为了单个应用程序设计的

2. 关系数据库模型

- RDB不支持：
  - 对象类型
  - 关联概念（如类之间的继承或方法）
  - 结构化类型
  - 集合
  - 引用
- RDB模型的主要原始类型是由多列组成的**关系表**。表的列只能取**原子值**——结构化值或值的集合是不允许的。
- 关系数据库的全貌：

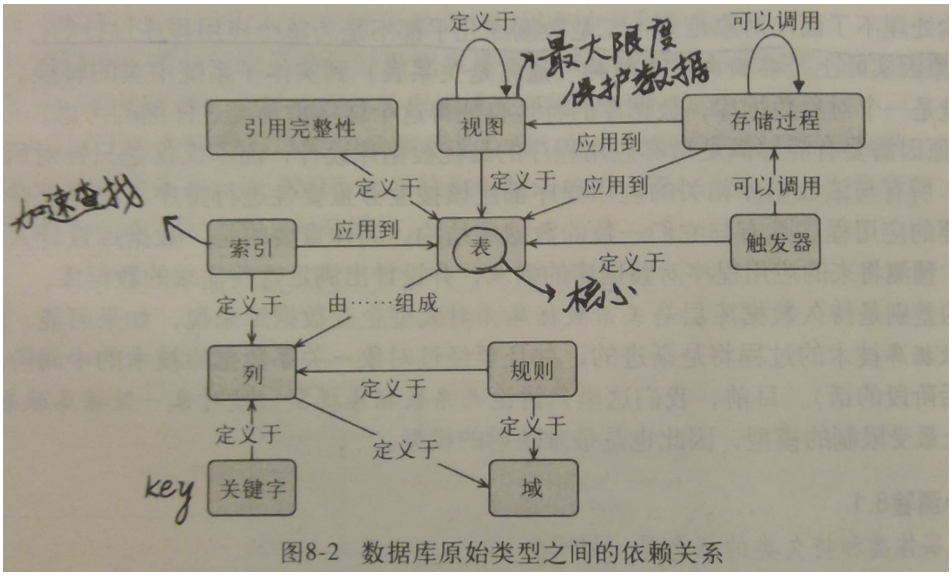
- 

图8-2 数据库原始类型之间的依赖关系

  - 关系型数据库的核心：表

a. 列、域和规则
   - 关系数据库采用列和行组成的表来定义数据。存储在任何列和行交叉处的数据值必须是简单的（不可分割的）和单一的（不重复的）值，我们说这样的列具有**原子域**（数据类型）
   - 只涉及单个列或域的较简单的业务规则可以在**规则**机制中定义，涉及多个表的更复杂的规则可以定义为**引用完整性约束**，定义业务规则的最佳机制是**触发器**

   i. 列
      1）关系型数据库采用由列和行组成的表来定义数据
      2）具有单一定义域，具有原子域（数据类型）
   ii. 域
      1）定义了一个列可以取值的合法集
   iii. 业务规则：
      1）默认值
      2）取值范围
      3）值列表
      4）值大小写
      5）值格式

b. 表
   i. 定义固定列的集合，行集合可变
   ii. 外键连接到其他表
   iii. 允许NULL值
   iv. 每个表要有一个主键（主关键字），关键字是列的最小集合，这些列的值能够唯一确定表中的一行。

c. 引用（参照）完整性
   - 当被参照表的数据发生变化时，参照表如何更新以保持数据的完整一致性

   i. 通过引用完整性约束来维护表间的关系
   ii. 外键是指表中的一组列，它的值要么是NULL，要么必须与同一个或另一个表中的主键值匹配。主键与外键的这种对应关系称为**引用完整性**

iii.

- Upd(R)；Del(R)：限制更新或删除操作（即如果Employee中还有行链接到这个Department行，就禁止操作）。
- Upd(C)；Del(C)：级联操作（即删除所有链接的Employee行）。
- Upd(N)；Del(N)：设置为空值（即更新或删除Department行，然后将链接的Employee行的dept_id设置为null）。
- Upd(D)；Del(D)：设置为缺省值（即更新或删除Department行，然后将链接的Employee行的dept_id设置为缺省值）。

iv. R，restrict如果employee表中的字段还有指向department的字段，则限制department的更新
C，cascade级联操作,department的字段改了，employee有关字段跟着改
N，NULL设置空值
D，default设置默认值

v. cpa规则，如果department的字段删除，employee的相关字段改嫁，链接到别的

d. 触发器
   i. 一个小程序，用扩展的SQL语句编写，当定义了触发器的表发生修改操作时自动执行（触发）
   ii. 触发器是一种不能被调用的特殊存储过程，当对一个表执行insert,update,delete操作时自我触发。这就意味着每个表可以有3个触发器

e. 存储过程
   i. 存储过程必须指定名字，可以有输入、输出参数
   ii. 执行效率比传统查询高，性能的优势在于缓解了**网络堵塞**
   iii.

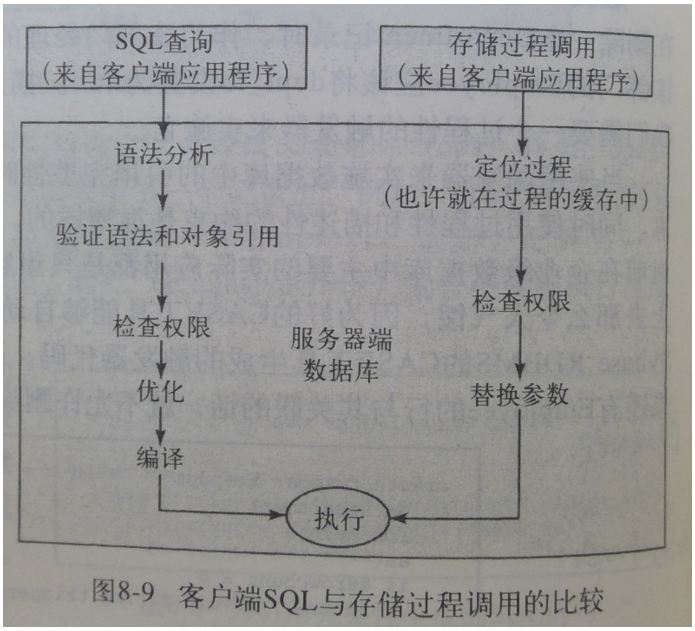


图8-9 客户端SQL与存储过程调用的比较

f. 视图——用来**支持数据库安全**
   i. 视图的本质是一条select语句，把这条语句起了个名
   ii. 关系视图是被存储和命名的SQL查询
   iii. 视图并不存储持久数据，只是将数据的快照展现出来
   iv. 不要对视图做非select操作，即不支持视图更新

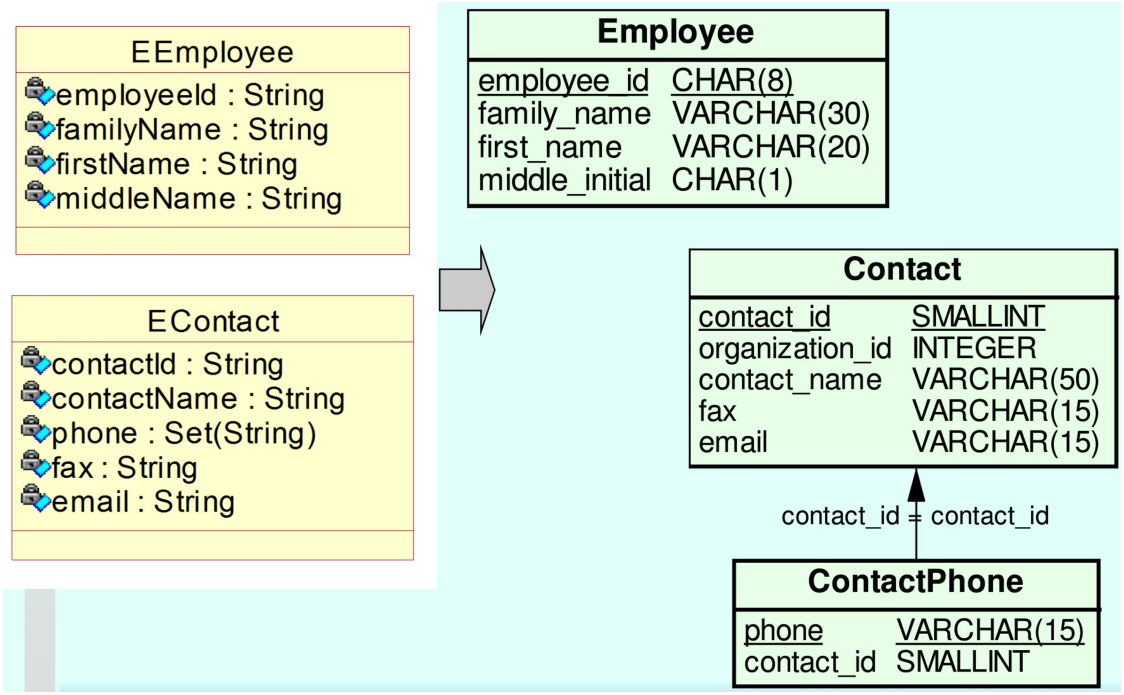
g. 范式
   - 高范式的表能满足所有低范式
   - 低范式的表可能会出现更新异常——对表进行修改操作所引起的意外的连带后果

   i. 第一范式
   ii. 第二范式

iii. 第三范式
iv. BCNF范式
v. 第四范式
vi. 第五范式

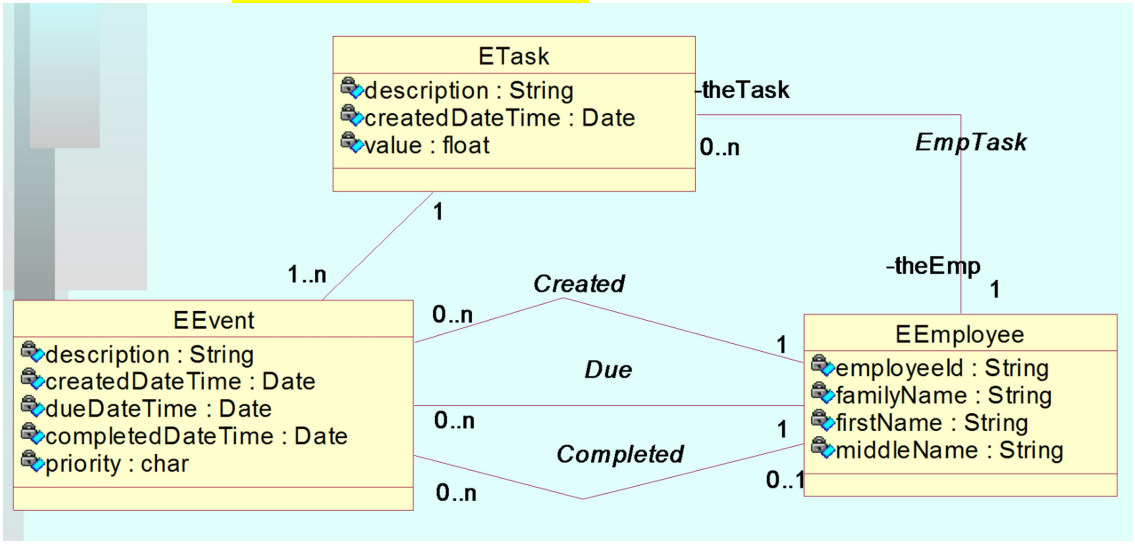
3. 对象-关系映射
   - 对任何一对一或者一对多的关联可以通过直接在一个表中插入一个外键以匹配另一个表的主键来实现
   - 对于多对多的关系，通过建立交叉表
   - 指从UML类模型映射到RDB模式的设计，把类图映射到数据库的模式设计

   a. 映射实体类
      i. 实体类到关系表的映射必须满足表的第一范式，列必须是原子的

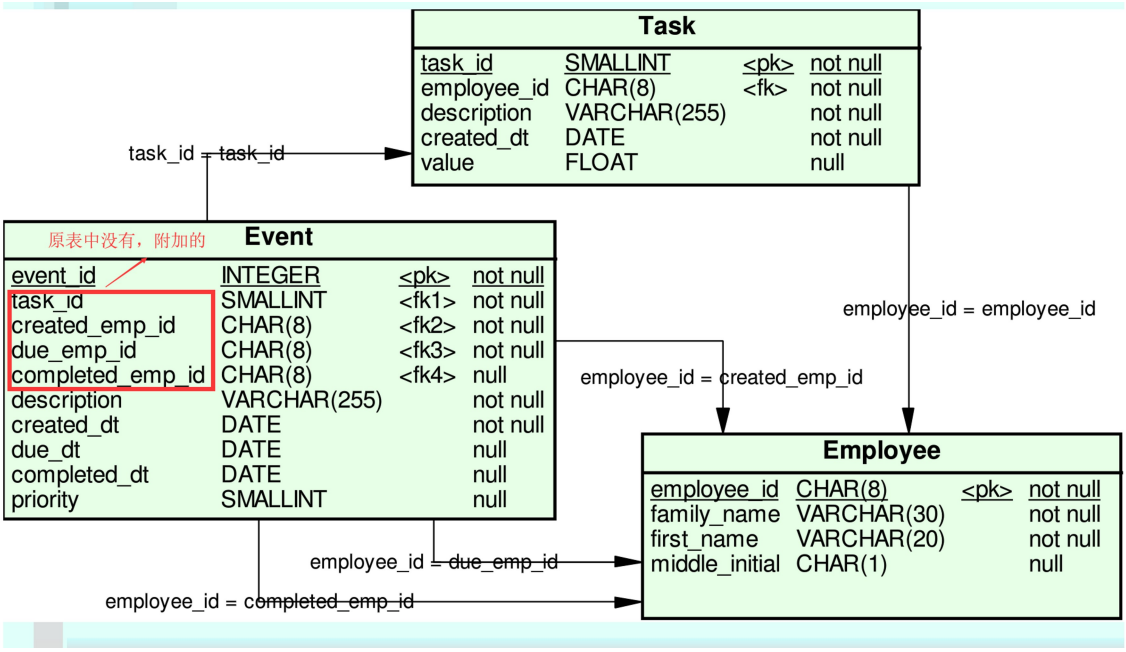
      ii.


      iii. 如果类图中涉及集合类型，映射时将集合字段属性拆开单独成表，如上的一个雇员有多个电话号码，将电话号码单独成表

   b. 映射关联
      i. 在多对一的关联中，在多的一端加一个外键关联别人

      ii.


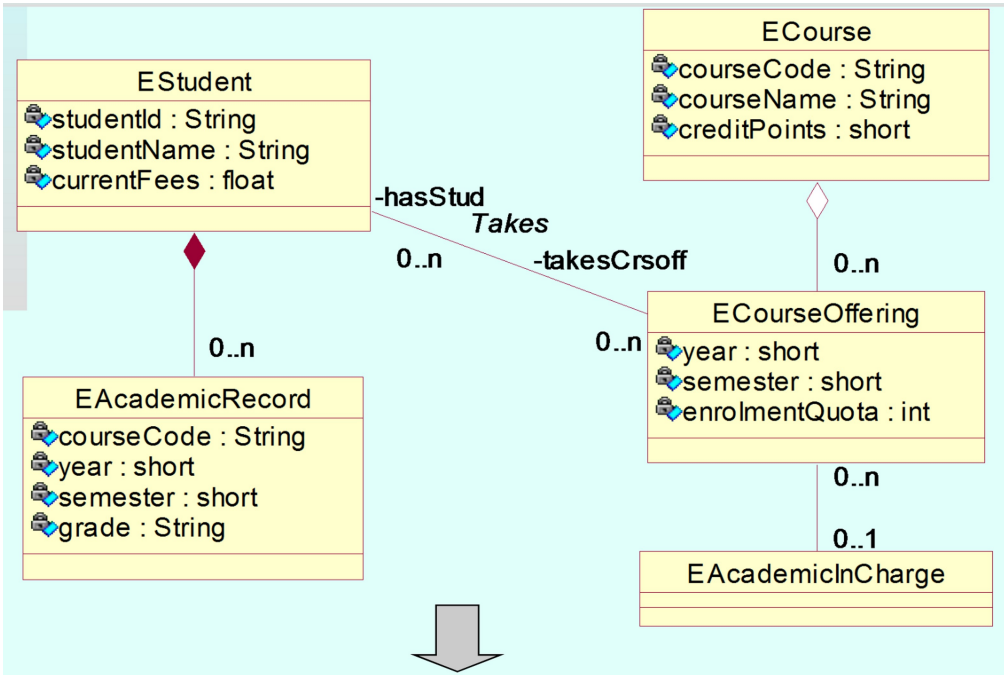
iii.



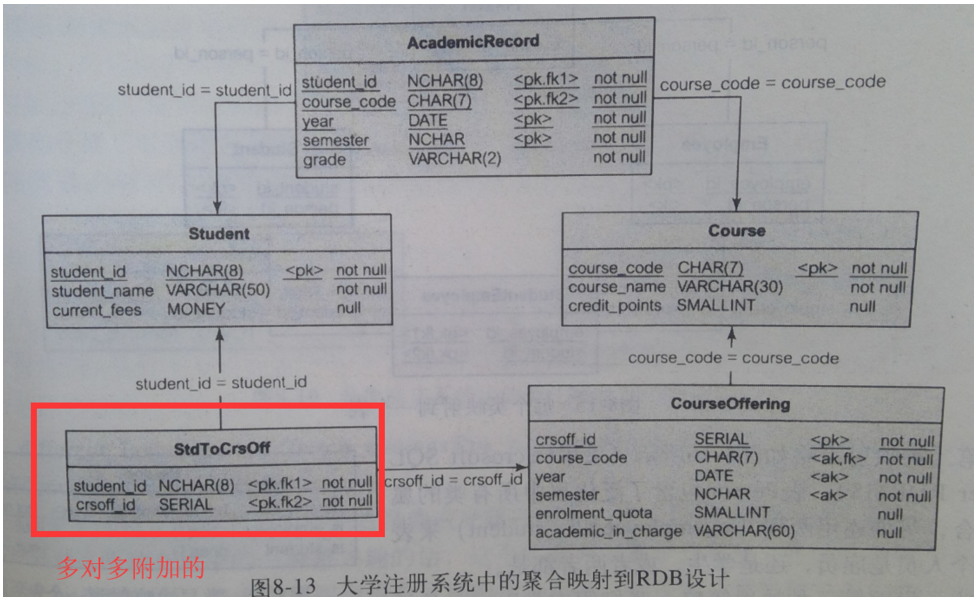
c. 映射聚合

i. 将聚合定义为一对多的关联

ii.

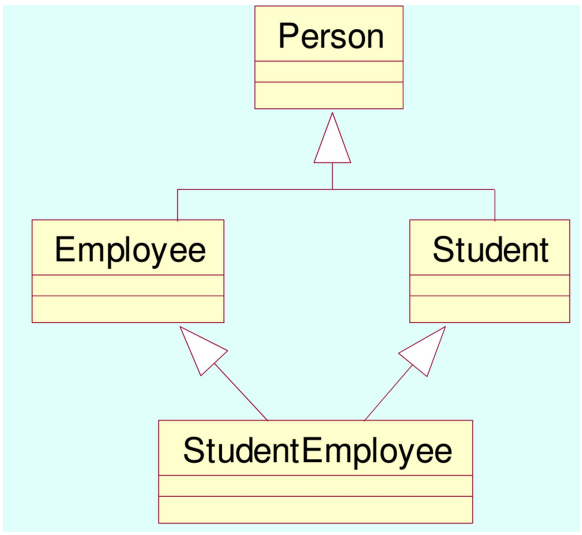


iii.



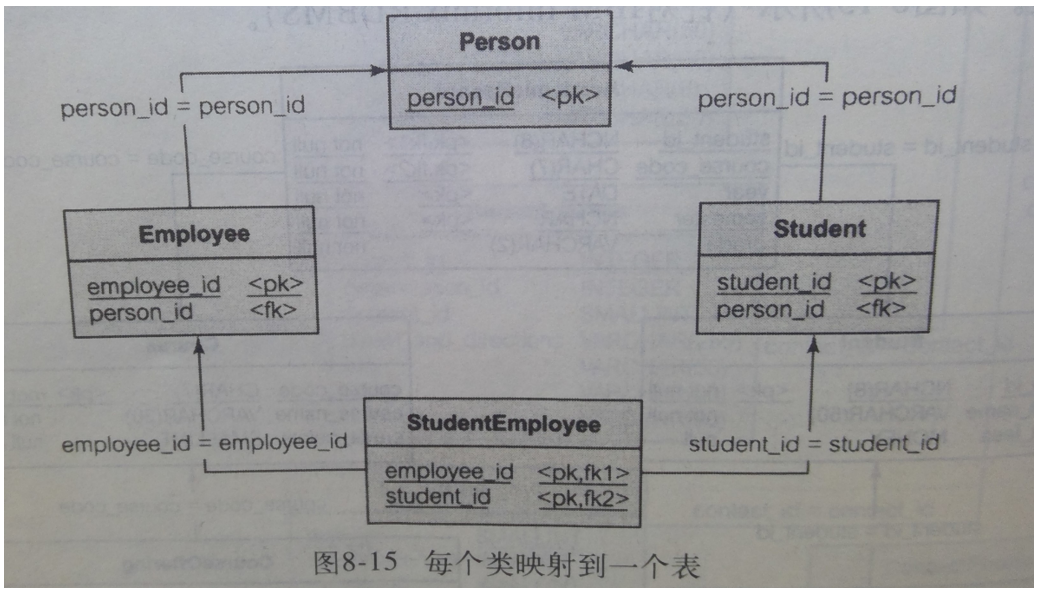
d. 映射泛化

i.



ii. 粗暴的方法：父类一，子类多

1)



iii. 第二种策略：映射到一个表，用两列来表示一个人是雇员还是学生还是二者都是

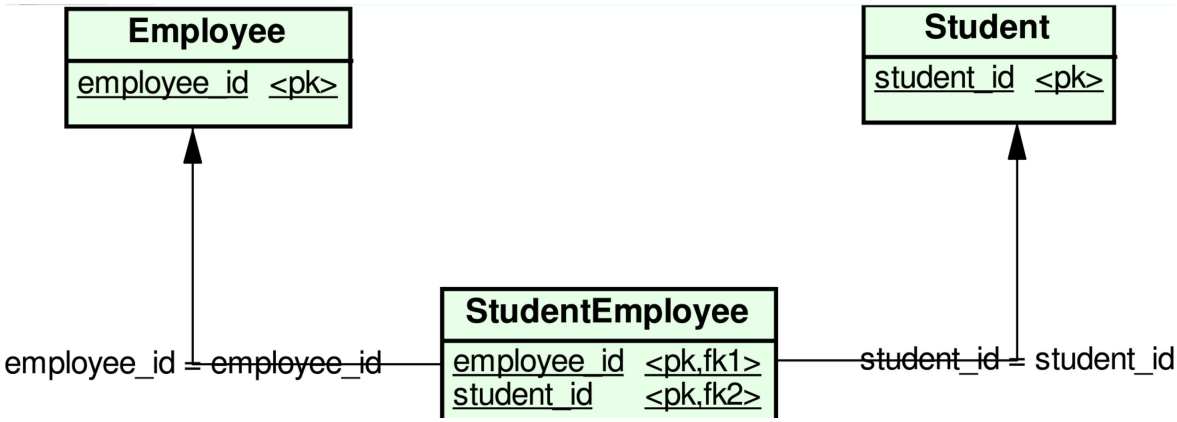
1) 管理成本降低，但是处理成本升高（查找效率变低）

2)

Person			
person_id	uniqueidentifier	<pk>	not null
is_employee	char(1)		null
is_student	char(1)		null

iv. 第三种策略：类person的所有属性都可以由具体类所对应的表“继承”，employee与student两个类的并集包含person

1)



v. 第四种策略：假设总是知道一个雇员是否是学生，设置boolean类型为not

NULL

1)

Employee			
employee_id	NCHAR(8)	<pk>	not null
is_student	BOOLEAN		not null

Student			
student_id	NCHAR(10)	<pk>	not null
is_employee	BOOLEAN		not null

4. 管理持久对象的模式
   - 企业应用体系结构设计（PEAA）用于解决持久对象的问题，其中一些模式包括：
     - 标识映射Identity Map：给内存中所有持久对象都指定对象标识符（OID），没在内存中的对象没有管
       -

OID	Address
所有的在内存中的对象	

     - 数据映射data mapper：程序随时都知道所需要的对象是在内存中还是从数据库中获得，具有当前内存中的对象的相关信息
       - 如果对象在内存中，那么数据映射确定它是否是**清洁**的——即它在内存中的状态（数据内容）与在数据库表中的对应记录是不是同步。如果对象是**脏**的，那么数据映射从数据库重新获取数据。
       - 判断一个对象是否清洁的相关信息可保存在数据映射中，但将这个信息保存在标识映射中或实体对象本身中会更好。
       - 如果对象在内存中，在**标识映射表**去找，如果不在则去加载进内存
     - 延迟装载lazy load：一个对象并没有包含你需要的所有数据，但知道如何获取
       - 只装载一部分对象到内存中
     - 工作单元unit of work：程序知道一个业务事务都包含内存中的哪些对象
       - 他负责协调改变的写入和解决并发问题
       - 和事务配套，随时可以回滚

   a. 检索持久对象

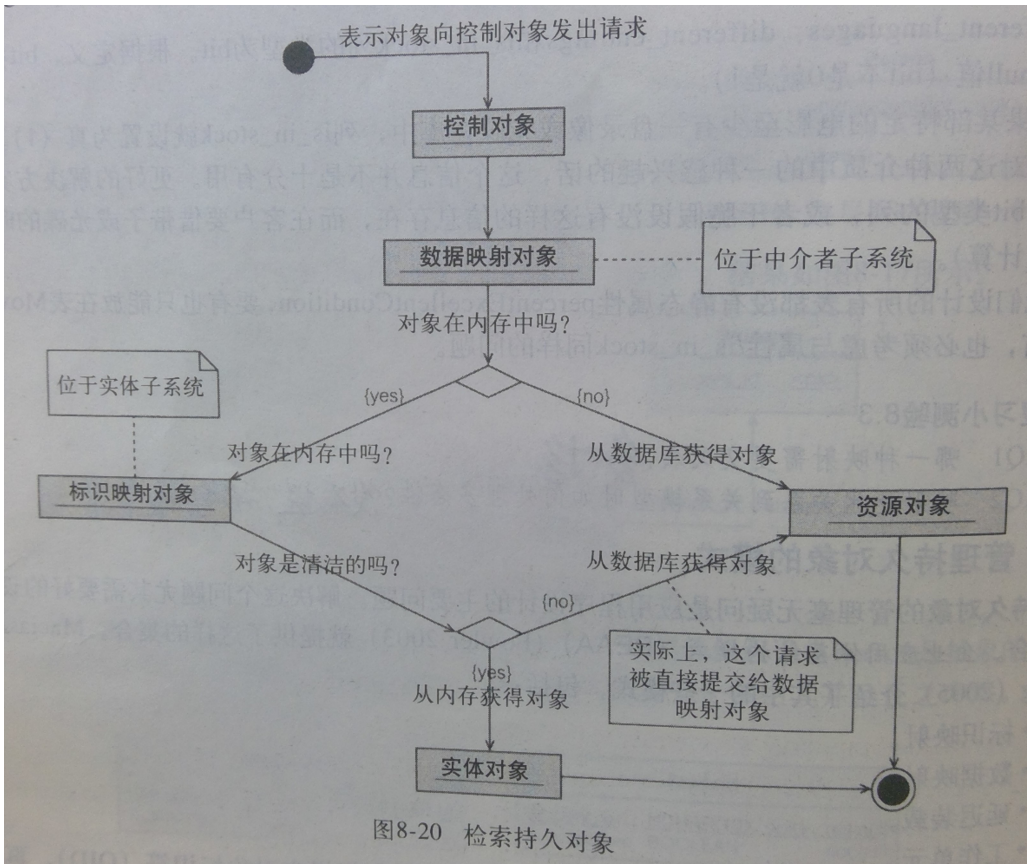
      i. 

图8-20 检索持久对象

   b. 装载持久对象

   c. 释放持久对象

      i. 释放（或检入）是装载的反操作

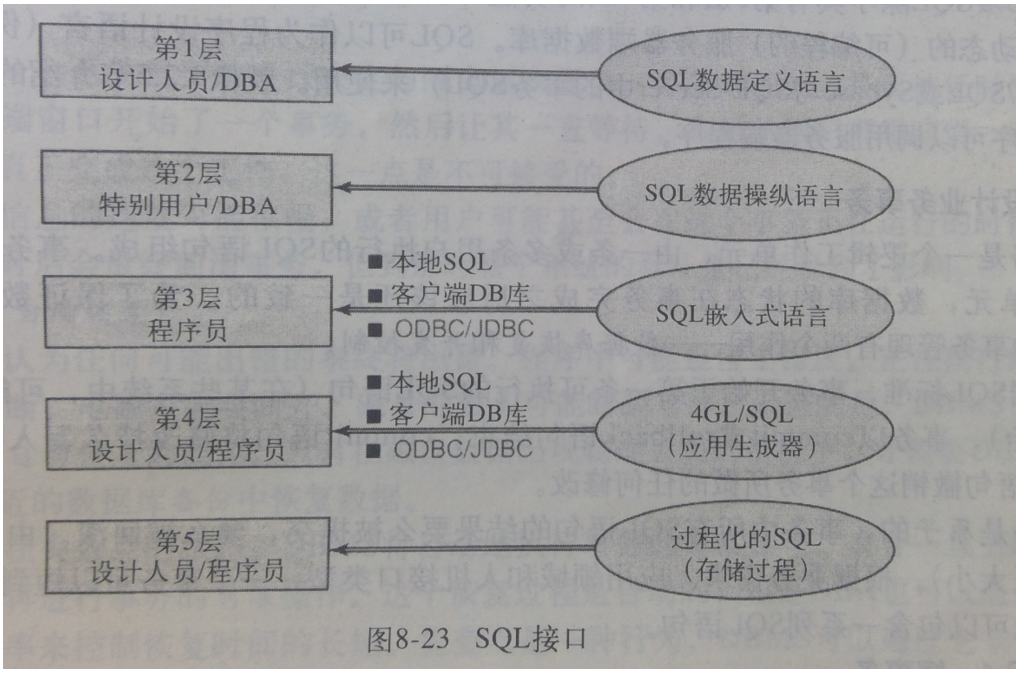
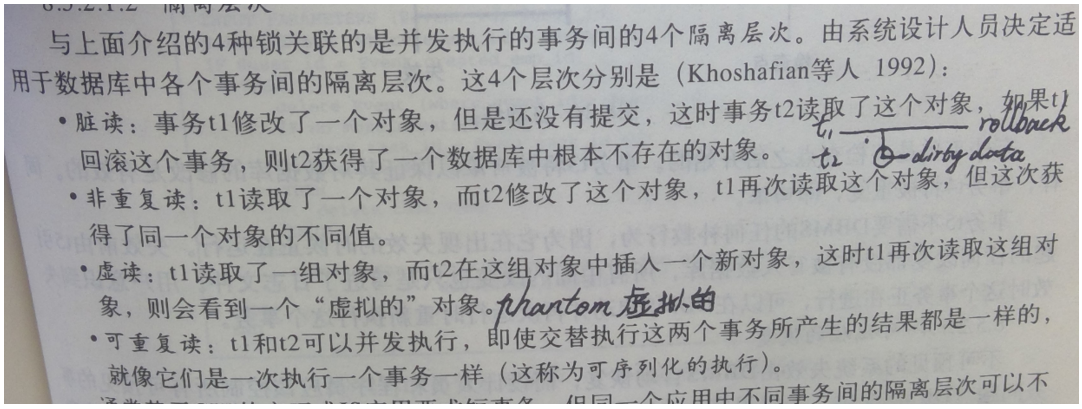
5. 设计数据库访问和事务
   a. SQL程序设计的层次
      i. 

图8-23 SQL接口
      ii. 第一层：DDL
      iii. 第二层：DML，应用系统程序员使用第二层以上的SQL，SQL除了允许像第二层一样一次一个集合的处理机制意外，还允许一次一个记录的处理机制
      iv. 第三层：被嵌入到常规的程序设计语言中，一个语句嵌入
      v. 第四层：将SQL嵌入到客户端程序中，第四代语言（4GL）
      vi. 第五层：存储过程
   b. 设计业务事务
      i. 事务是一个逻辑工作单元，保证数据库的一致性
      ii. DBMS事务管理有两个作用——**数据库恢复和并发控制**
      iii. 事务开始于第一条可执行SQL语句，以commit或rollback语句结束。commit语句将修改持久写入数据库中，rollback撤销这个事务所作的任何修改。
      iv. 锁：
         1) 排他（写）锁：必须等待当前持有锁的事务释放该锁之后才能处理其他事务
         2) 更新（预写）锁：其他事务可以读取对象，但有需要可以随时升级为排他锁
         3) 读（共享）锁：其他事务可以读取对象并可能得到这个对象的更新锁
         4) 无锁：其他事务可以随时更新对象，因此它只适用于**允许脏读**的应用系统，即一个事务读取的数据可能在该事务完成之前被另一个事务修改或删除了
      v. 隔离层次：
         1) 脏读
         2) 非重复读
         3) 虚读
         4) **可重复读**（保证事务的可序列化的执行）

5)

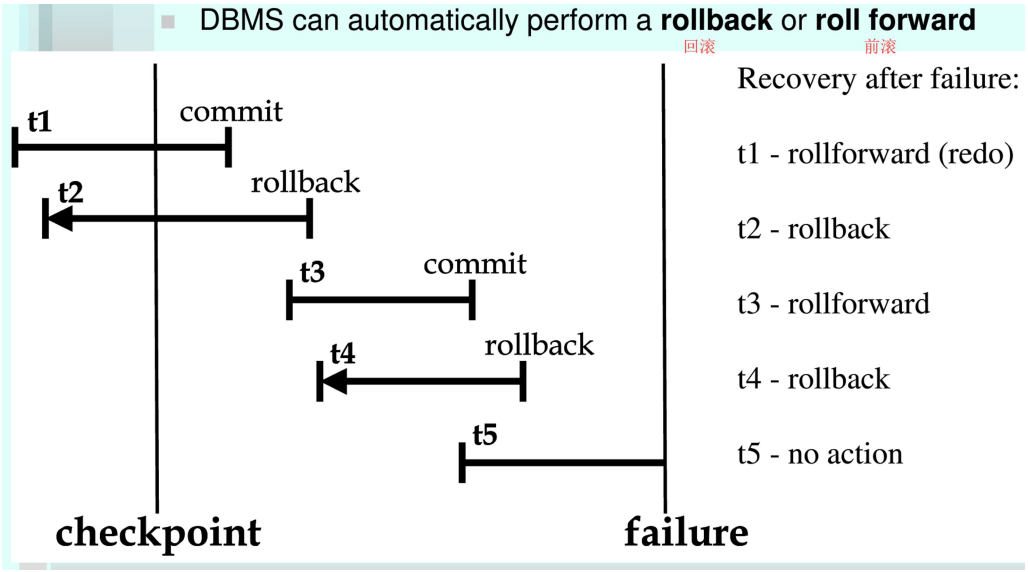


与上面介绍的4种锁关联的是并发执行的事务间的4个隔离层次。由系统设计人员决定适用于数据库中各个事务间的隔离层次。这4个层次分别是（Khoshafian等人 1992）：

- 脏读：事务t1修改了一个对象，但是还没有提交，这时事务t2读取了这个对象，如果t1回滚这个事务，则t2获得了一个数据库中根本不存在的对象。
- 非重复读：t1读取了一个对象，而t2修改了这个对象，t1再次读取这个对象，但这次获得了同一个对象的不同值。
- 虚读：t1读取了一组对象，而t2在这组对象中插入一个新对象，这时t1再次读取这组对象，则会看到一个“虚拟的”对象。
- 可重复读：t1和t2可以并发执行，即使交替执行这两个事务所产生的结果都是一样的，就像它们是一次执行一个事务一样（这称为可序列化的执行）。

vi. 自动恢复

□



□ 如果事务没被提交，恢复可以rollback

□ 如果事务被提交后，恢复需要做一个逆操作

1) 可编程的恢复：
   a) 保存点，是程序中的一条语句，把一个长事务分成若干个短事务
   b) 触发器回滚，是一种特殊的保存点，允许用触发器回滚事务
2) 存储过程